

Q1. Tell me about yourself.

Best answer for you:

"I'm a Full Stack Developer with over 3 years of experience developing end-to-end CRM and SaaS applications. My primary technology stack is React.js, Node.js, Express.js and MySQL.

In my current role at Creative Ethics in Hyderabad, I've been involved in the complete development lifecycle, from requirement gathering and system design to development, deployment and production support.

My major project is a Real Estate CRM platform where I worked on Lead Management, Property Management, Customer Management, Loan Management, HRM, Inventory, Payroll and Project Management modules.

On the technical side, I've developed REST APIs, implemented JWT authentication and role-based authorization, optimized database queries, built responsive React dashboards, and integrated third-party services.

One of the key integrations I worked on was Meta Facebook and Instagram Lead Ads using webhooks to synchronize leads into the CRM in real time. I also implemented duplicate lead detection and automatic lead merging.

I've worked with AWS services such as S3, Amplify, Route 53 and CloudFront, and I've also handled production deployments and troubleshooting.

I'm now looking for an opportunity where I can work on larger-scale applications and deeper integrations involving APIs, cloud infrastructure, payments and enterprise systems, which is why this Y-Axis opportunity interests me."

This answer is strongly supported by your resume.

ROUND 2 — YOUR CRM PROJECT

Q2. Explain your Real Estate CRM architecture.

Answer:

"The application follows a frontend-backend-database architecture.

On the frontend, we use React.js with reusable components and Material-UI for the UI.

The frontend communicates with the backend through REST APIs.

The backend is built using Node.js and Express.js. I separated the application into routes, middleware, controllers and database-related logic.

JWT is used for authentication, and role-based authorization controls what different users can access.

MySQL is used as the primary database. We optimized the schema and indexes for better query performance.

External services such as Meta Lead Ads and email services communicate with our backend through APIs and webhooks.

The application is deployed in a production Linux environment and I have also worked with AWS services."

Your resume supports this overall architecture and production responsibility.

Q3. What modules did you develop?

Answer:

"The CRM contains Lead Management, HRM, Employee Management, Attendance, Payroll, Inventory and Project Management. The main Real Estate CRM also includes Property Management, Customer Relationship Management, Loan Management, quotations, receipts, invoices, payments, document management, notifications and analytics dashboards."

Your resume explicitly lists these modules.

ROUND 3 — REST API

Q4. How do you design a REST API?

Answer:

"First I identify the resource and define the API contract. Then I create appropriate HTTP methods and endpoints.

For example, for leads I could have:

POST /leads
GET /leads
GET /leads/:id
PUT /leads/:id
DELETE /leads/:id

I validate incoming data, authenticate the request using JWT middleware, authorize the user's role, execute the business logic, interact with the database and return an appropriate HTTP status code.

I also handle errors centrally and use pagination, filtering and sorting for list APIs."

Your resume specifically mentions REST APIs, validation, pagination, filtering and optimized queries.

Q5. What status codes do you commonly use?

Know these:

Code	Meaning
200	Successful request
201	Resource created
400	Bad request
401	Authentication required/failed
403	Authenticated but not authorized
404	Resource not found
409	Conflict/duplicate
422	Validation error
500	Server error

ROUND 4 — JWT + RBAC

Q6. Explain how JWT authentication works in your CRM.

Answer:

"When the user logs in, the backend validates the credentials. If they're valid, the server generates a JWT containing the required user information and sends it to the client.

For subsequent protected API requests, the client sends the token. Middleware verifies the JWT signature and extracts the user information.

After authentication, I check the user's role and permissions before allowing access to protected resources.

For example, an Admin may have access to all CRM modules, while a Sales Executive may only access the resources permitted to that role."

Your resume specifically states JWT authentication, protected routes and role-based authorization.

Q7. Authentication vs authorization?

Authentication:

"Who are you?"

Authorization:

"What are you allowed to access?"

Example:

```
Login
↓
Authentication
↓
JWT
↓
Authorization
↓
Check Role
↓
Allow/Deny
```

ROUND 5 — WEBHOOK / META INTEGRATION

This is potentially your **strongest interview topic** because it directly overlaps with their requirement for webhook and third-party integrations.

Q8. Explain your Facebook/Instagram Lead Ads integration.

Answer:

"We integrated Meta Lead Ads with our CRM using webhooks for real-time lead synchronization.

When a lead is submitted through Facebook or Instagram, Meta sends a webhook event to our backend.

Our backend receives and validates the webhook request. From the event, we identify the relevant lead information and then use the Meta API to retrieve the required lead details.

We validate the data and check whether the lead already exists in our CRM. If it's a new lead, we create the record. If it's a duplicate, we apply our duplicate detection and merging logic.

This allows leads from Meta to enter the CRM automatically without manual data entry."

This is directly supported by your resume.

Q9. What is a webhook?

Answer:

"A webhook is a mechanism where one system sends an HTTP request to another system automatically when a specific event occurs. Unlike polling, our system doesn't continuously ask whether something changed. The external system pushes the event to our endpoint."

Q10. What happens if the same webhook is received twice?

Strong answer:

"I would make the webhook processing idempotent. I would identify the event or business record using a unique identifier and check whether it has already been processed before creating a new record.

For lead synchronization, I would also use unique identifiers such as the external lead ID and combine that with normalized phone or email checks to prevent duplicate CRM records."

This connects directly to your documented duplicate-detection work.

ROUND 6 — DATABASE

Q11. How did you improve database performance?

Answer:

"I first identify the slow APIs and determine whether the bottleneck is in the database query.

Then I inspect the query execution, joins and filtering conditions. I optimize the query and add appropriate indexes based on frequently searched or joined columns.

I also avoid retrieving unnecessary data and use pagination for large datasets.

In my project, these optimizations helped reduce API response time by around 30%."

The 30% improvement is explicitly stated in your resume.

Q12. What is an index?

"An index is a database data structure that helps the database locate records faster without scanning the entire table."

Then give a CRM example:

```
leads
id
name
phone
email
status
assigned_to
created_at
```

Frequently searching:

```
phone
email
assigned_to
status
```

could potentially benefit from appropriate indexes.

ROUND 7 — NEXT.JS

This is a **gap area**, so don't claim production experience unless you have it.

Q13. You have React experience. What's the difference between React and Next.js?

Answer:

"React is primarily a UI library for building component-based interfaces. Next.js is a framework built around React that provides additional application-level capabilities such as routing, server-side rendering, static generation, server components and backend functionality.

My production experience has primarily been with React.js, but the React foundation makes the transition to Next.js straightforward. I'm familiar with the major Next.js concepts and architecture."

Q14. SSR vs CSR?

CSR

Browser downloads JavaScript and renders the page.

SSR

Server generates HTML and sends it to the browser.

SSG

Pages are generated ahead of time.

Interview answer:

"CSR is useful for highly interactive applications such as internal CRM dashboards. SSR can improve initial page loading and SEO for public-facing pages. SSG is useful for pages where the content doesn't change frequently."

ROUND 8 — SALESFORCE

This is another **JD gap**.

Q15. Have you worked with Salesforce?

If you haven't, **do not say yes**.

Use:

"Salesforce has not been part of my production stack so far. My production integration experience has primarily been with REST APIs, webhooks and Meta integrations. However, I understand the integration pattern for Salesforce REST APIs, OAuth 2.0 and SOQL, and I'm comfortable working with third-party APIs because I've already implemented similar integration architectures."

That's much safer than pretending.

Q16. How would you integrate Salesforce with your CRM?

Answer:

```
CRM
↓
Node.js backend
↓
OAuth 2.0
↓
Salesforce Access Token
↓
Salesforce REST API
↓
Create / Update Lead
↓
Store Salesforce ID
↓
CRM Database
```

Mention:

- OAuth2
- access/refresh token handling
- API limits

- retries
 - duplicate prevention
 - error handling
 - synchronization
-

ROUND 9 — PAYMENT GATEWAYS

Again, don't claim production experience if you haven't actually integrated one.

Q17. Explain payment gateway integration.

Answer:

"The backend should create a payment order or transaction request with the gateway. The frontend uses the gateway's payment flow. After payment, the gateway provides a callback or webhook.

The backend should verify the transaction using the gateway's server-side verification mechanism rather than trusting the frontend response.

Once verified, we update our internal transaction record.

We also need to handle failures, retries, duplicate callbacks, refunds and reconciliation."

Q18. What if payment succeeds but your server doesn't receive the callback?

Excellent answer:

"I wouldn't rely only on the callback. I would use the gateway's transaction-status API as a fallback and maintain a reconciliation mechanism.

The transaction-processing endpoint should also be idempotent so that if the same successful payment is processed multiple times, we don't create duplicate transactions."

That's exactly the type of thinking this Y-Axis role is looking for.

ROUND 10 — AWS

Q19. Which AWS services have you worked with?

Your resume says:

"I've worked with AWS S3, Amplify, Route 53 and CloudFront."

Be prepared to explain each:

S3

Object/file storage.

CloudFront

CDN for delivering content closer to users.

Route 53

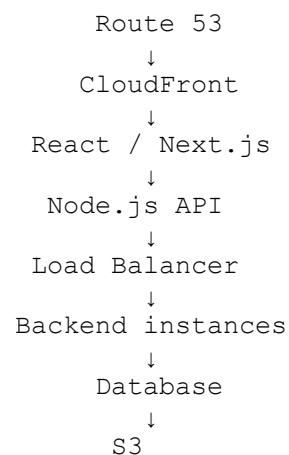
DNS/domain routing.

Amplify

Hosting/deployment platform for web applications.

Q20. How would you deploy your CRM to AWS?

A reasonable architecture:



Then mention:

- HTTPS
- environment variables/secrets
- logging
- monitoring
- backups
- scaling

ROUND 11 — DOCKER

Q21. What is Docker?

"Docker packages an application together with its dependencies into a container so that it can run consistently across environments."

Know:

```
Dockerfile
  ↓
Image
  ↓
Container
```

Q22. Image vs container?

Image:

Blueprint/package.

Container:

Running instance of that image.

ROUND 12 — KUBERNETES / EKS

Q23. What is Kubernetes?

"Kubernetes is a container orchestration platform used to deploy, manage, scale and recover containerized applications."

Know these terms:

Pod
Deployment
Service
Ingress
ConfigMap
Secret
Namespace

Q24. What is EKS?

"Amazon EKS is AWS's managed Kubernetes service."

Again, if you haven't used EKS professionally, say:

"I understand the architecture and concepts, but my production deployment experience has primarily been outside EKS."

ROUND 13 — MICROSERVICES

Q25. Monolith vs microservices?

Monolith

One application

- Users
- Leads
- Payments
- Notifications
- Reports

Microservices

API Gateway

- User Service
- Lead Service
- Payment Service
- Notification Service
- Reporting Service

Answer:

"A monolith keeps functionality in one deployable application, while microservices split functionality into independently deployable services. Microservices can provide independent scaling and deployment, but they introduce additional complexity around communication, monitoring, deployment and distributed data."

ROUND 14 — JAVASCRIPT

These are almost guaranteed for a Full Stack role.

Q26. `let`, `const`, `var`?

Know:

```
var    → function scoped
let    → block scoped
const → block scoped, cannot be reassigned
```

Q27. What is a closure?

"A closure occurs when a function retains access to variables from its outer lexical scope even after the outer function has completed."

Q28. What is the event loop?

Explain:

```
Call Stack
  ↓
Web/Node APIs
  ↓
Callback / Microtask Queue
  ↓
Event Loop
  ↓
Call Stack
```

ROUND 15 — REACT

Q29. `useEffect` VS `useMemo`?

`useEffect`

Used for side effects.

Examples:

- API calls
- subscriptions
- timers

`useMemo`

Used to memoize a calculated value.

Q30. How do you optimize React?

Mention:

- reusable components
- memoization when appropriate
- pagination
- lazy loading
- code splitting
- avoiding unnecessary state
- avoiding unnecessary re-renders
- optimized API calls

Your resume already highlights reusable components and frontend performance optimization.

ROUND 16 — SECURITY

Q31. How do you secure a REST API?

Give this answer:

"I would use HTTPS, JWT or OAuth2 for authentication, role-based authorization, input validation, parameterized queries, secure password hashing, appropriate CORS configuration,

rate limiting, secure headers, proper secret management and centralized error handling. I would also avoid exposing sensitive information in responses and logs."

Q32. Why bcrypt?

"bcrypt is a password hashing algorithm designed to securely hash passwords with a computational cost that makes brute-force attacks more difficult."

Your resume specifically lists bcrypt.

ROUND 17 — PRODUCTION SCENARIOS

These are **very important** for a 3–5 year developer.

Q33. Production API suddenly becomes slow. What do you do?

Answer:

1. Check monitoring/logs
 2. Identify affected endpoint
 3. Check DB performance
 4. Check external APIs
 5. Check CPU/memory
 6. Check recent deployments
 7. Analyze slow queries
 8. Optimize
 9. Test
 10. Deploy
 11. Monitor
-

Q34. API works locally but gives 500 in production.

Check:

Environment variables
Database connection
Credentials
CORS
File permissions
Node version

Dependencies
Production configuration
Logs
Network/security rules

ROUND 18 — YOUR PROJECT DEEP DIVE

Q35. What was the most challenging feature?

A good choice from your resume is the **Meta integration + duplicate detection**.

You can say:

"One challenging area was integrating external lead sources into the CRM reliably. The system had to receive leads in real time, process the webhook, retrieve the required information, validate it and prevent duplicate records. We implemented duplicate detection and automatic merging so that leads coming from multiple marketing channels didn't create unnecessary customer records."

Your resume explicitly documents this functionality.

ROUND 19 — CODING ROUND

Practice these **15 problems**:

Easy

1. Reverse a string
2. Find largest number
3. Find second largest
4. Remove duplicates from array
5. Count character frequency
6. Check palindrome
7. Find missing number

JavaScript

8. Implement debounce
9. Implement throttle
10. Explain/implement Promise behavior

- 11. Flatten nested array
- 12. Group array objects by property

Backend

- 13. Create a REST API for Leads
- 14. Implement JWT middleware
- 15. Implement duplicate-lead detection

ROUND 20 — HR

Q36. Why do you want to leave your current company?

Don't criticize your employer.

Answer:

"I've gained strong hands-on experience in full-stack development and CRM systems in my current role. At this stage, I'm looking for an opportunity where I can work on larger-scale systems, enterprise integrations and more advanced cloud and backend architecture. This role interests me because it combines full-stack development with Salesforce, payments, APIs and AWS."

Q37. Why Y-Axis?

Answer:

"Y-Axis operates at significant scale and the role involves exactly the kind of engineering work I'm interested in—full-stack applications, CRM integrations, REST APIs, payment systems and AWS infrastructure. My existing experience in CRM development, API integrations and webhook-based lead synchronization gives me a strong foundation, and I see this role as an opportunity to grow into larger enterprise systems."

Q38. Why should we hire you?

Strong answer:

"I bring hands-on experience building a production CRM end to end rather than only working on isolated features. I've worked across frontend, backend, APIs, database optimization, authentication, third-party integrations, deployment and production support.

I've also worked directly with business stakeholders to convert business requirements into technical solutions.

The areas I already have strong experience in—React, Node.js, Express, REST APIs, webhooks, authentication, CRM workflows and AWS—align well with the core of this position. I'm also comfortable learning the additional enterprise technologies in the role such as Salesforce, payment platforms, Kubernetes and Jenkins."